
Financial Mathematics: Bond Market and Beyond

A Practical Commentary — From Theory to the Trading Desk

Alessandro Cirillo

April 24, 2026

Contents

1	Introduction and Motivation	3
2	Market Assumptions and the Absence of Arbitrage (AOA)	3
2.1	The Three Pillars	3
2.2	Zero-Coupon Bond (ZCB) Constraints	3
3	Interest Rates — From Prices to Rates	4
3.1	Spot and Forward Rates	4
3.2	Instantaneous Rates	4
4	Coupon Bonds — Pricing via AOA	5
4.1	Fixed Coupon Bond	5
4.2	Floater (Positive Index-Link)	6
4.3	Reverse Floater (Negative Index-Link)	6
5	Swap Contracts	7
5.1	Interest Rate Swap (IRS)	7
6	Immunization — Protecting the Portfolio Against Rate Moves	8
6.1	Duration	8
6.2	Convexity	8
6.3	Immunization Theorems	9
7	Defaultable Bonds and Credit Spreads	10
7.1	Pricing Framework	10

8	Credit Derivatives	11
8.1	Asset Swap Spread	11
8.2	Credit Default Swap (CDS)	11
9	Conclusions	13
10	Summary Table	14

1 Introduction and Motivation

This document accompanies the computational project on the bond market. Its goal is twofold: to explain the *mathematical meaning* of each quantity computed in the code, and to anchor every concept to a *concrete problem* that arises daily on a fixed-income trading desk.

2 Market Assumptions and the Absence of Arbitrage (AOA)

2.1 The Three Pillars

The entire code rests on three assumptions:

1. **Frictionless market:** no transaction costs, taxes, or short-selling constraints; securities are infinitely divisible.
2. **Competitive market:** agents maximise profit and are price-takers.
3. **Absence of Arbitrage (AOA):** there is no strategy with non-positive cost today and a strictly positive payoff in at least one future state.

Definition 2.1 (Arbitrage). *A strategy $\mathbf{x}/\mathbf{t}$ is a*

- forward arbitrage if $X_0 \leq 0$, $X_k \geq 0$ for all k , and $\exists j : X_j > 0$;
- spot arbitrage if $X_0 < 0$ and $X_k \geq 0$ for all k .

2.2 Zero-Coupon Bond (ZCB) Constraints

To ensure AOA, a unit zero-coupon bond $B(t, s)$, which pays 1 at maturity s and is priced at t , must satisfy:

$$B(t, s) > 0, \quad B(s, s) = 1, \quad B(t, s) < 1 \text{ whenever } t < s \text{ and rates are positive.} \quad (1)$$

Trading Desk Scenario — Mispricing Detection

A junior trader notices that the price of a 2-year ZCB is *higher* than that of a 1-year ZCB issued by the same sovereign. According to the AOA theorem on decreasingness with respect to maturity ($B(t, s_1) > B(t, s_2)$ for $s_1 < s_2$), this is impossible in an arbitrage-free market. The code implements this check automatically:

1. Fetch the current term structure of ZCB prices from the Bloomberg terminal.
2. Verify monotonicity: flag any inversion as a potential arbitrage.
3. If a genuine inversion exists (e.g., due to a liquidity squeeze in a specific maturity bucket), the desk's arbitrage desk can execute the strategy and lock in a riskless profit.

3 Interest Rates — From Prices to Rates

3.1 Spot and Forward Rates

Given a ZCB price $B(t, T)$, the code computes four equivalent representations:

$$\text{Spot rate: } i(t, T) = B(t, T)^{-\frac{1}{T-t}} - 1 \quad (2)$$

$$\text{Spot yield: } h(t, T) = -\frac{1}{T-t} \ln B(t, T) \quad (3)$$

$$\text{Forward rate: } i(t, s, T) = B(t, s, T)^{-\frac{1}{T-s}} - 1 \quad (4)$$

$$\text{Forward yield: } h(t, s, T) = -\frac{1}{T-s} \ln B(t, s, T) \quad (5)$$

where $B(t, s, T) = B(t, T)/B(t, s)$ is the *implied (forward) ZCB price*, guaranteed by the AOA Implied Prices Theorem.

Python — Building the Term Structure

```

1 import numpy as np
2
3 def spot_yield(B, t, T):
4     """Continuously-compounded spot yield."""
5     return -np.log(B) / (T - t)
6
7 def forward_zcb_price(B_t_T, B_t_s):
8     """Implied forward ZCB price (AOA Implied Prices
9         Theorem)."""
10    return B_t_T / B_t_s
11
12 def forward_yield(B_t_s_T, s, T):
13     """Continuously-compounded forward yield."""
14     return -np.log(B_t_s_T) / (T - s)
15
16 # Example: 3-month, 6-month and 1-year ZCB prices (as
17 # of today, t=0)
18 B = {0.25: 0.9900, 0.50: 0.9780, 1.00: 0.9560}
19
20 for tau, price in B.items():
21     print(f"h(0,{tau}) = {spot_yield(price, 0, tau)
22         :.4%}")

```

3.2 Instantaneous Rates

The instantaneous forward rate is defined as:

$$f(t, u) = -\frac{\partial \ln B(t, u)}{\partial u} \quad (6)$$

From this, the ZCB price is recovered as:

$$B(t, T) = \exp\left(-\int_t^T f(t, u) du\right) \quad (7)$$

and the spot yield is simply the average of instantaneous forwards:

$$h(t, T) = \frac{1}{T-t} \int_t^T f(t, u) du \quad (8)$$

Trading Desk Scenario — Yield Curve Construction (Bootstrapping)

Every morning the rates desk *bootstraps* the yield curve from liquid instruments (OIS, government bonds, IRS). The code performs exactly this: starting from observed market prices, it extracts the term structure of spot yields $\{h(0, T_k)\}_{k=1}^n$ and instantaneous forward rates $f(0, T)$. The forward curve is then used to price any non-standard maturity instrument that is not directly quoted in the market, for instance, a corporate bond maturing in 7 years and 3 months.

4 Coupon Bonds — Pricing via AOA

4.1 Fixed Coupon Bond

By the *Linear Price Theorem*, any coupon bond is a portfolio of ZCBs. If a bond pays periodic coupon c/m at dates $t_1, \dots, t_n$ and reimburses C'_T at maturity T , its no-arbitrage price is:

$$P_t = \frac{c}{m} \sum_{i=1}^n B(t, t_i) + C'_T B(t, T) \quad (9)$$

Python — Fixed Coupon Bond Pricing

```

1 def coupon_bond_price(coupon_rate, nominal, m,
2   maturities, zcb_prices, reimbursement):
3     """
4     Price a fixed coupon bond using the AOA linear
5     price theorem.
6     coupon_rate : annual coupon rate (e.g. 0.05 for 5%)
7     nominal      : face value
8     m            : coupon frequency (e.g. 2 for semi-
9                   annual)
10    maturities   : list of coupon dates [t1, t2, ..., tn]
11    zcb_prices   : dict {maturity: B(0, maturity)}
12    reimbursement: C'_T (usually = nominal)
13    """
14    periodic_coupon = nominal * coupon_rate / m

```

```

12 coupon_pv = sum(periodic_coupon * zcb_prices[t] for
    t in maturities)
13 redemption_pv = reimbursement * zcb_prices[
    maturities[-1]]
14 return coupon_pv + redemption_pv

```

4.2 Floater (Positive Index-Link)

A *Floater* pays coupons indexed to a market rate. The key insight is that each indexed coupon can be replicated by a portfolio of ZCBs:

$$B(t, t_1) - B(t, t_2) = B(t, t_2) i(t, t_1, t_2) \quad (10)$$

With a liquidity spread s , the AOA Floater price (with market convention) is:

$$P_t = (1 + c) B(t, t_1) + s \sum_{i=1}^n B(t, t_i) \quad (11)$$

Trading Desk Scenario — CCT Pricing and the Corrective Spread

Italian CCTs (Certificati di Credito del Tesoro) are sovereign floaters indexed to BOT rates. A government bond desk needs to price them daily. The code reproduces the theoretical floater price and compares it to the observed market price. The difference, captured by the spread s , is the *corrective spread*: it reflects liquidity premia and any deviation between the BOT curve and the OIS/swap curve used for discounting. Monitoring this spread over time is a risk signal: a sudden widening may indicate deteriorating market liquidity or a credit event.

4.3 Reverse Floater (Negative Index-Link)

A Reverse Floater pays $c_k = \bar{c} - \alpha i(t, t_{k-1}, t_k)$: the coupon *decreases* when rates rise. The AOA price is:

$$P_t = \bar{c} \sum_{k=1}^n B(t, t_k) - \alpha (1 - B(t, t_n)) + B(t, t_n) \quad (12)$$

Risk Warning — Reverse Floater Convexity

A reverse floater exhibits an inverse sensitivity to interest rates, often resulting in an effective negative duration over certain rate ranges. Its price tends to increase when rates fall, making it attractive for investors expecting rate cuts. However, due to its nonlinear payoff structure and high convexity, unexpected rate increases can lead to significant losses. For this reason, the code flags this non-standard duration behavior and incorporates a convexity-adjusted risk assessment.

5 Swap Contracts

5.1 Interest Rate Swap (IRS)

An IRS exchanges a *fixed leg* (at rate r^S) for a *floating leg*. The swap rate that sets the contract value to zero at inception is:

$$r^S = \frac{1 - B(t_0, t_n)}{\sum_{i=0}^{n-1} B(t_0, t_{i+1})} \quad (13)$$

Python — Swap Rate Calculation

```

1 def swap_rate(zcb_prices, payment_dates):
2     """
3     Compute the par swap rate from the term structure.
4     zcb_prices      : dict {date: B(0, date)}
5     payment_dates: [t1, t2, ..., tn]
6     """
7     B_tn = zcb_prices[payment_dates[-1]]
8     annuity = sum(zcb_prices[t] for t in payment_dates)
9     return (1 - B_tn) / annuity
10
11 def forward_swap_rate(zcb_prices, payment_dates,
12                       start_index):
13     """Forward-start swap rate (exchange begins at
14     payment_dates[start_index])."""
15     B_tj = zcb_prices[payment_dates[start_index]]
16     B_tn = zcb_prices[payment_dates[-1]]
17     annuity = sum(zcb_prices[t] for t in payment_dates[
18                   start_index:])
19     return (B_tj - B_tn) / annuity

```

Trading Desk Scenario — Hedging a Fixed-Rate Loan Book

A corporate bank has granted five-year fixed-rate loans and funded them with short-term deposits (floating rate). It is exposed to a rise in floating rates. The solution is to *enter a payer IRS* (pay fixed, receive floating): the floating receipts from the swap offset the deposit cost. The code computes the fair swap rate r^S using Eq. (13) and sizes the notional to match the loan book exposure.

6 Immunization — Protecting the Portfolio Against Rate Moves

6.1 Duration

Definition 6.1 (Generalized Duration). *Given a bond paying $\mathbf{x}/t$ and term structure S :*

$$D^\circ(t, \mathbf{x}, S) = \frac{\sum_{k=1}^n (t_k - t) x_k B(t, t_k)}{\sum_{k=1}^n x_k B(t, t_k)} \quad (14)$$

Duration is a *price-weighted average time* to cash flows. The modified (volatility) duration is $D^M = D/(1 + i)$, and:

$$\frac{dV}{V} \approx -D^M di \quad (15)$$

Python — Duration and Modified Duration

```

1 def duration(cash_flows, dates, zcb_prices, t=0):
2     """Generalized (Macaulay) duration."""
3     weights = [(tk - t) * xk * zcb_prices[tk]
4                for tk, xk in zip(dates, cash_flows)]
5     price = sum(xk * zcb_prices[tk]
6                for tk, xk in zip(dates, cash_flows))
7     return sum(weights) / price
8
9 def modified_duration(dur, spot_rate):
10    """Volatility of price with respect to rate changes
11    ."""
12    return dur / (1 + spot_rate)
13
14 def dv01(price, mod_dur):
15    """Dollar Value of 1 basis point (DV01)."""
16    return price * mod_dur * 0.0001

```

6.2 Convexity

Definition 6.2 (Generalized Convexity).

$$C^\circ(t, \mathbf{x}, S) = \frac{V''(t, \mathbf{x}, S)}{V(t, \mathbf{x}, S)} = \frac{\sum_{k=1}^n (t_k - t)^2 x_k B(t, t_k)}{\sum_{k=1}^n x_k B(t, t_k)} \quad (16)$$

The second-order Taylor expansion for a price change under a parallel shift $\Delta\delta$ is:

$$\Delta V \approx -D^\circ V \Delta\delta + \frac{1}{2} C^\circ V (\Delta\delta)^2 \quad (17)$$

6.3 Immunization Theorems

Theorem 6.3 (Fisher-Weil). *If $V_0(A) = V_0(L)$, $D_A = D_L$, and $C_A \geq C_L$, then a parallel shift of the yield curve cannot decrease the portfolio value $V(A) - V(L)$.*

Theorem 6.4 (Redington). *If liabilities span multiple maturities, $V_0(A) = V_0(L)$, $D_A = D_L$, and $C_A \geq C_L$, then the portfolio is immunized against any parallel shift.*

Theorem 6.5 (Fong-Vasicek). *For non-parallel shifts parameterized by $Y(T)$, the relative portfolio change satisfies $\Delta V/V \geq K(C_A - C_L)$, where K depends on the slope change. If the shift is convex ($K > 0$), maximise convexity; if concave ($K < 0$), minimise it.*

Python — Immunization Check (Redington)

```

1 def immunization_check(assets, liabilities, zcb_prices)
2     :
3     """
4     Verify Redington immunization conditions.
5     assets, liabilities: list of (cash_flow, date)
6     tuples
7     Returns: dict with value_match, duration_match,
8             convexity_ok
9     """
10    dates_A, flows_A = zip(*assets)
11    dates_L, flows_L = zip(*liabilities)
12
13    V_A = sum(x * zcb_prices[t] for x, t in zip(flows_A,
14    , dates_A))
15    V_L = sum(x * zcb_prices[t] for x, t in zip(flows_L,
16    , dates_L))
17
18    D_A = duration(flows_A, dates_A, zcb_prices)
19    D_L = duration(flows_L, dates_L, zcb_prices)
20
21    # Second-order duration
22    C_A = sum((t**2) * x * zcb_prices[t] for x, t in
23    zip(flows_A, dates_A)) / V_A
24    C_L = sum((t**2) * x * zcb_prices[t] for x, t in
25    zip(flows_L, dates_L)) / V_L
26
27    return {
28        "value_match"      : abs(V_A - V_L) < 1e-4,
29        "duration_match"   : abs(D_A - D_L) < 1e-3,
30        "convexity_ok"    : C_A >= C_L,
31        "surplus_convexity": C_A - C_L
32    }

```

Trading Desk Scenario — Insurance Company ALM

An insurance company has long-duration liabilities (pension payouts in 20–30 years) funded by a bond portfolio. The ALM (Asset-Liability Management) desk uses immunization theory daily:

1. Compute D_L from the actuarial liability schedule.
2. Build an asset portfolio so that $D_A = D_L$ and $C_A \geq C_L$.
3. Run the `immunization_check` function each morning; if conditions are broken (e.g. because bond prices moved overnight), the desk rebalances by trading government bonds or IRS.
4. The $DV01$ (dollar value of a basis point) computed from modified duration tells the desk exactly how many notional of bonds must be bought or sold.

7 Defaultable Bonds and Credit Spreads

7.1 Pricing Framework

A defaultable bond uses a *defaultable term structure* $B^*(t, T)$ incorporating credit risk. Its price is:

$$DP_t = \sum_{i=1}^n c B^*(t, t_i) + B^*(t, t_n) \quad (18)$$

It can be decomposed as:

$$DP_t = P_t - H, \quad H = \text{DefP}(t, T) = B(t, T) \cdot EL \quad (19)$$

where $EL = \mathbb{E}[LGD \mathbf{1}_{\tau < T}]$ is the expected loss and $LGD = 1 - RR$ is the loss given default.

The *credit spread* term structure measures the extra yield demanded for bearing credit risk:

$$cs(t, T) = h^*(t, T) - h(t, T) \quad (20)$$

Python — Credit Spread Extraction

```

1 def credit_spread(B_star, B, t, T):
2     """
3     Extract the credit spread from defaultable vs risk-
4     free ZCB prices.
5     B_star : defaultable ZCB price B*(t,T)
6     B       : risk-free ZCB price B(t,T)
7     """
8     h_star = -np.log(B_star) / (T - t) # defaultable
9     h       = -np.log(B) / (T - t)     # risk-free
10    return h_star - h                  # credit

```

```

10         spread in bps * 1e4
11 def expected_loss(credit_spread, tenor, recovery_rate
12     =0.40):
13     """
14     Approximate EL from credit spread (flat hazard rate
15     assumption).
16     PD      cs / LGD      (from cs      PD * LGD for small
17     probabilities)
18     """
19     lgd = 1 - recovery_rate
20     pd  = 1 - np.exp(-credit_spread * tenor) #
21         cumulative PD
22     return pd * lgd

```

Trading Desk Scenario — Corporate Bond Relative Value

A credit analyst on a corporate bond desk screens the investment-grade universe for *relative value*. For each issuer, the code:

1. Bootstraps the risk-free (OIS/swap) curve.
2. Computes the implied credit spread $cs(t, T)$ for each bond.
3. Derives the implied probability of default (PD) and expected loss (EL) using a flat hazard rate.
4. Compares implied PD to the rating agency's historical default rate: if the market prices in a PD much higher than historical evidence, the bond may be *cheap* (buy signal).

8 Credit Derivatives

8.1 Asset Swap Spread

An asset swap bundles a defaultable bond with an IRS. The spread that equates the fixed leg to the floating leg is:

$$s_{\text{ASW}} = c - r^S + \frac{1 - DP_t}{\sum_{i=1}^n B(t, t_i)} \quad (21)$$

A zero spread implies the bond is fairly priced relative to the swap curve. A positive spread signals underpricing relative to the default-free world (credit risk, liquidity, or tax).

8.2 Credit Default Swap (CDS)

A CDS is the standardized insurance contract against default:

- **Premium leg:** the protection buyer pays $s \cdot \mathbf{1}_{t_{i-1} < \tau}$ at each t_i until default or maturity.
- **Protection leg:** the seller pays $LGD \cdot \mathbf{1}_{t_{i-1} < \tau \leq t_i}$ upon default.

The fair CDS spread s is found by equating the expected present values of the two legs:

$$s = \frac{\sum_{i=1}^n B(0, t_i) LGD \mathbb{P}(t_{i-1} < \tau \leq t_i)}{\sum_{i=1}^n B(0, t_i) \mathbb{P}(\tau > t_{i-1})} \quad (22)$$

Python — CDS Fair Spread (Flat Hazard Rate)

```

1 def cds_spread(hazard_rate, recovery_rate,
2   payment_dates, zcb_prices, lgd=None):
3     """
4     Compute the fair CDS spread under a constant hazard
5     rate lambda.
6     Survival probability: P(tau > t) = exp(-lambda * t)
7     """
8     if lgd is None:
9         lgd = 1 - recovery_rate
10
11     protection_leg = 0.0
12     premium_annuity = 0.0
13
14     prev_date = 0.0
15     for t in payment_dates:
16         surv_beg = np.exp(-hazard_rate * prev_date)
17         surv_end = np.exp(-hazard_rate * t)
18         dt = t - prev_date
19
20         # Expected loss paid at t_i if default occurred
21         # in (t_{i-1}, t_i]
22         default_prob = surv_beg - surv_end
23         protection_leg += zcb_prices[t] * lgd *
24             default_prob
25
26         # Premium paid if survived to t_{i-1}
27         premium_annuity += zcb_prices[t] * surv_beg
28         prev_date = t
29
30     return protection_leg / premium_annuity
31
32 # Example: 5Y CDS on a BBB-rated corporate
33 lambda_hat = 0.02 # 2% annual hazard rate
34 RR = 0.40 # 40% recovery
35 tenors = [0.5 * k for k in range(1, 11)] # semi-
36     annual, 5 years
37 B_0_t = {t: np.exp(-0.035 * t) for t in tenors} #

```

```

flat 3.5% OIS curve
33
34 s_cds = cds_spread(lambda_hat, RR, tenors, B_0_t)
35 print(f"Fair CDS spread: {s_cds * 10000:.1f} bps")

```

Trading Desk Scenario — CDS vs. Bond Basis Trade

The *CDS–bond basis* is the difference between the CDS spread and the asset swap spread of the same issuer:

$$\text{Basis} = s_{\text{CDS}} - s_{\text{ASW}}$$

In a perfect, frictionless world, the basis should be zero (replication argument). In practice, funding costs, collateral rules, and supply/demand imbalances create a non-zero basis. The code:

1. Computes s_{CDS} and s_{ASW} daily for each name in the portfolio.
2. Signals a *negative basis trade* (buy bond + buy CDS protection) when the basis is unusually negative (bond is cheap relative to CDS protection).
3. Monitors the trade PnL as the basis converges back to zero.

This is a standard relative-value strategy run by credit macro desks.

9 Conclusions

The bond market is governed by a surprisingly small number of principles (primarily the Absence of Arbitrage) yet their implications are vast and immediately actionable on a trading desk. The code translates each mathematical object (zero-coupon prices, yields, durations, credit spreads) into functions that can be called in real time against live market data. The most important practical insight is that *no formula in this document requires any probability model or economic assumption*: all results follow purely from the requirement that no free lunch exists. When a model-free price is compared to the market price and a discrepancy is found, the difference reflects either a genuine market inefficiency (rare) or the presence of factors not yet included in the model, liquidity, taxes, counterparty risk, regulatory constraints. Quantifying and monitoring these spreads is the daily bread of any fixed-income professional.

10 Summary Table

Concept	Mathematical Object	Desk Application
AOA	$B(t, s_1) > B(t, s_2)$ for $s_1 < s_2$	Mispricing scanner
Term structure	$\{h(0, T_k)\}_{k=1}^n, f(0, T)$	Daily curve bootstrapping
Coupon bond	$P_t = \frac{c}{m} \sum B(t, t_i) + C'_T B(t, T)$	Bond valuation
Floater	Replication via ZCB portfolios	CCT pricing, spread
Swap rate	$r^S = \frac{1-B(t_0, t_n)}{\sum B(t_0, t_{i+1})}$	IRS hedging, ALM
Duration	$D^\circ = \sum w_k(t_k - t)$	DV01, rate sensitivity
Convexity	$C^\circ = V''/V$	Gamma risk, shifts
Redington	$D_A = D_L, C_A \geq C_L$	Insurance/pension ALM
Credit spread	$cs = h^* - h$	Relative value screening
CDS spread	Prot. leg = Prem. leg	CDS–bond basis trade

Table 1: Key quantities and their desk use cases.